

Section 1, Q2 — Single Source Shortest Path using MapReduce

Distributed Systems, Homework 3. Python 3 mappers and reducers, Bash orchestration, Slurm for the distributed runs.

1. Design

Dijkstra's algorithm needs a global priority queue and settles one vertex at a time, so there is nothing to divide between map tasks. Bellman-Ford divides: one relaxation round touches every edge and the work inside a round is independent. Each MapReduce job performs one round; a driver reruns jobs until no distance changes.

State is one line per node, keyed by node id:

```
node \t dist|v1,w1;v2,w2;...
```

Keeping the adjacency list inside the node's own record makes the job iterable — a reducer's output is valid input for the next round.

Map emits two kinds of pair per record:

Key	Value	Meaning
node	N dist adj	structure record
v (each out-neighbour)	D dist+w	candidate distance

— PROF The structure record is required, not an optimisation: a reducer only sees what mappers emit, so without it the graph would be gone after one round. It also carries the node's distance at the start of the round, which the reducer needs to detect an improvement. **D** records are emitted only when `dist < INF`, so the sentinel never propagates as `INF + w`.

Combine collapses every **D** record for the same node into a minimum on each mapper's own output — valid because min is associative and commutative. It cuts shuffle volume by 3× (measured, at P=1). The same pass also partitions its output by `node_id % R`, so every record for a node reaches the same reducer and R reducers can run concurrently. Partitioning inside the combiner avoids adding another process to the per-record path.

Reduce groups by node id, writes `min(old_dist, candidates)`, and increments a counter when a distance falls, emitted as `reporter:counter:SSSP,UPDATED,<n>`.

Termination: rerun while the reducers' **UPDATED** counters sum above zero, capped at V rounds. Zero updates means no edge can be relaxed, which for non-negative weights is the solution. Measured round counts were 14, 22 and 25 for the three graph sizes.

2. Compilation and execution

Nothing to compile — Python 3 and Bash only.

```
./sssp_local.sh tests/sample.txt output.txt 4          # single machine, 4
map tasks
sbatch --ntasks=4 sssp_slurm.sh big.txt big_out.txt    # cluster
for p in 1 2 4 8; do sbatch --ntasks=$p bench_slurm.sh; done  #
benchmark sweep
```

Input is **V E** then **E** lines of **u v w**; output is **node_id distance** sorted by node id with **INF** for unreachable nodes. Benchmark graphs come from **gen_graph.py V E seed** and are reproducible from the seed. Full instructions are in **README.md**.

3. Correctness

Verified against **dijkstra_ref.py**, a sequential Dijkstra with identical input and output format, so the two are compared with **diff**. **./verify.sh** runs both on the assignment sample (**0 0, 1 3, 2 2, 3 7**), a 5-node chain, a 3-node cycle, an edgeless graph, a disconnected graph, and three random 150-node graphs. Each case runs at several map-task counts and several reducer counts, because the answer must depend on neither: a mapper must not lose records at a chunk boundary, and the partitioner must not split one node's proposals across two reducers. All 54 cases pass.

PROF

The benchmark sweep repeats the check on every timed graph. All 21 measured configurations — three graph sizes, four process counts, one and R reducers — matched the sequential result exactly.

A stronger check than a diff is the iteration count, which was identical in every configuration: 14, 22 and 25 for the three graphs, whatever P and R were set to. If either parameter affected the computation, this number would move.

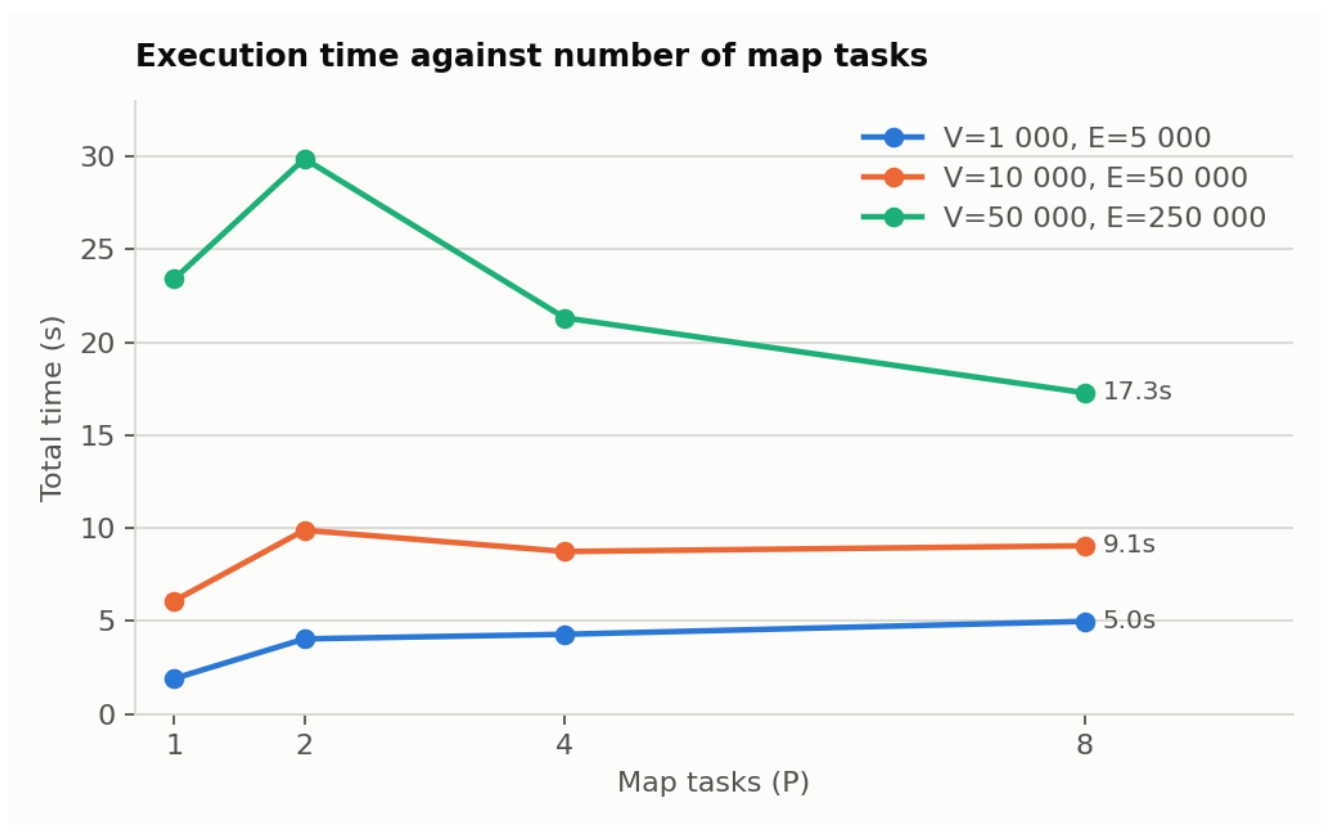
4. Performance

RCE **node06** (2 × Xeon Gold 6226R, 64 cores visible, 93 GB RAM), Python 3.6.8, seed 7. The eight jobs were serialised with **#SBATCH --dependency=singleton** so none competed for cores. Raw data in **perf_results/**.

Two configurations were measured at each process count: **R=1**, a single reducer, and **R=P**, one reducer per map task. At P=1 they are the same run.

Execution time

Graph	Reducers	P=1	P=2	P=4	P=8	Sequential Dijkstra
V=1 000, E=5 000	R=1	1.92 s	3.74 s	3.93 s	4.14 s	0.024 s
	R=P	1.92 s	4.05 s	4.29 s	4.99 s	
V=10 000, E=50 000	R=1	6.10 s	9.14 s	8.63 s	8.42 s	0.090 s
	R=P	6.10 s	9.90 s	8.75 s	9.06 s	
V=50 000, E=250 000	R=1	23.41 s	28.35 s	22.82 s	19.92 s	0.470 s
	R=P	23.41 s	29.88 s	21.33 s	17.29 s	



PROF

Multiple reducers help only the largest graph, where they cut 13% off the P=8 time. On the two smaller graphs they make things worse: each reducer is a fresh process started every iteration, and below a certain partition size that costs more than the parallelism returns.

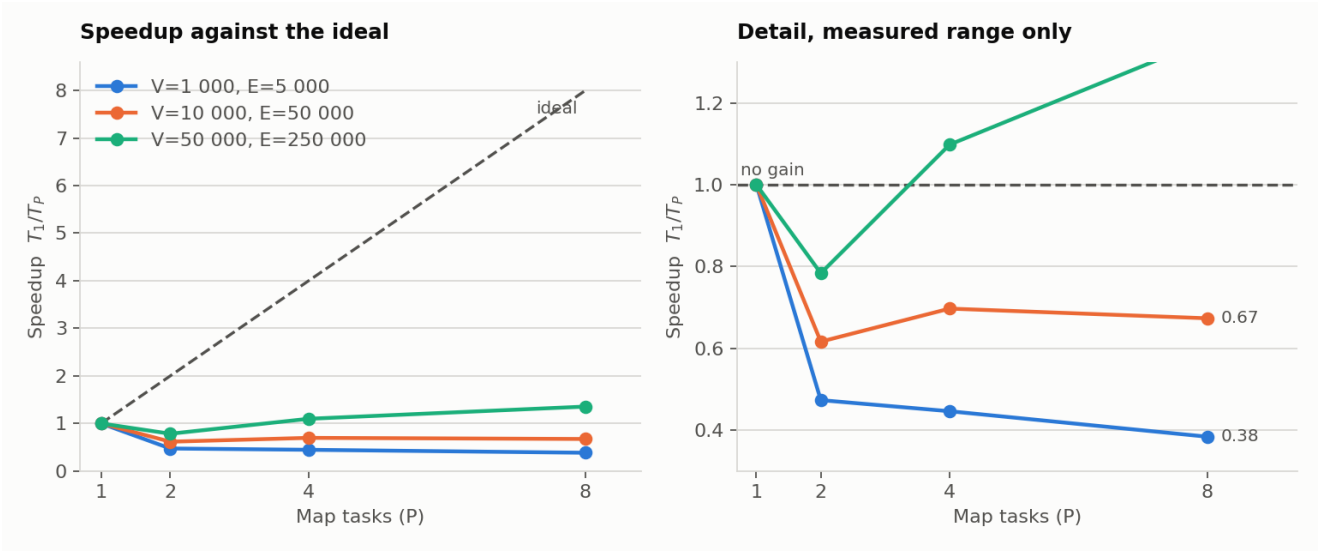
Speedup and efficiency

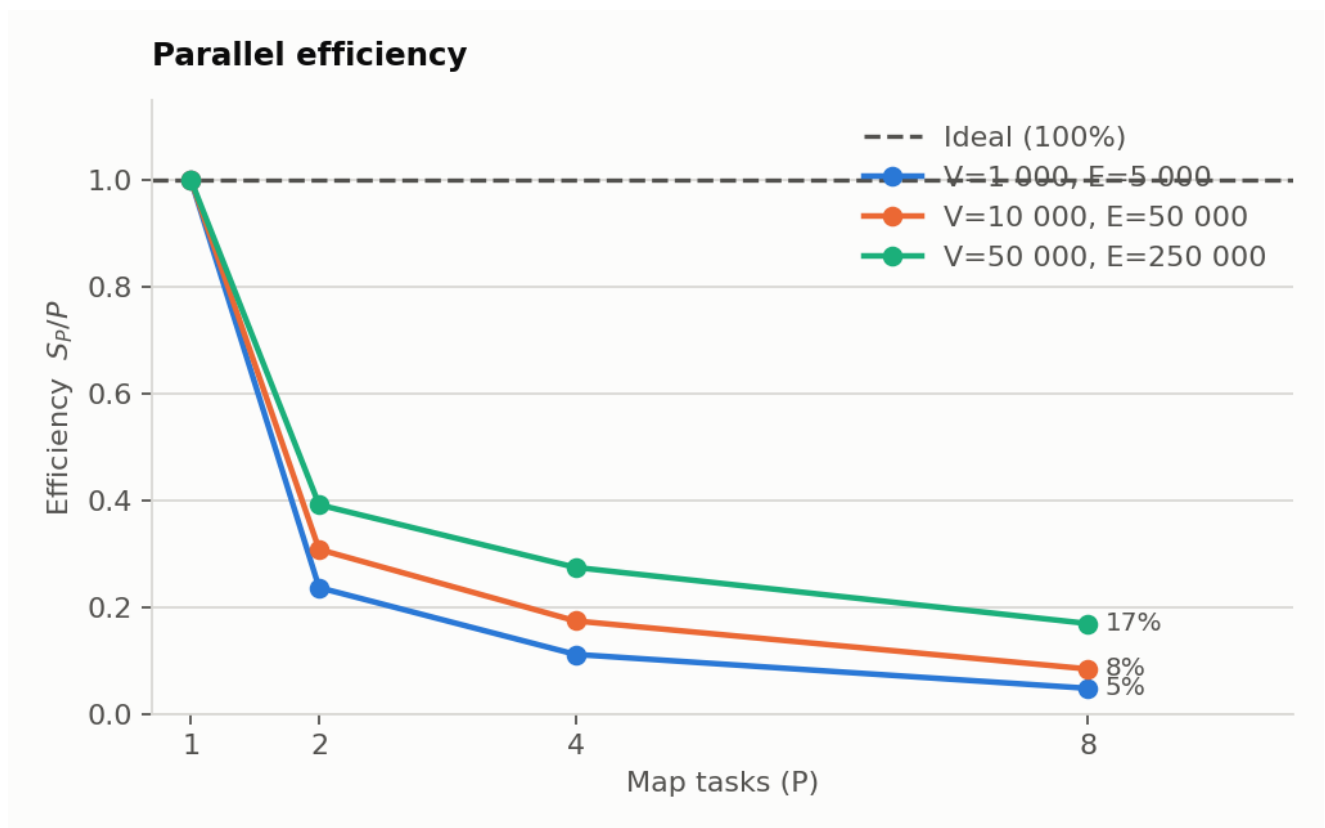
$S_P = T_1/T_P$, efficiency $\eta = S_P/P$, both relative to the single-task run.

Graph	Reducers	P=1	P=2	P=4	P=8
Speedup					
V=1 000, E=5 000	R=1	1.00	0.51	0.49	0.46

Graph	Reducers	P=1	P=2	P=4	P=8
	R=P	1.00	0.47	0.45	0.38
V=10 000, E=50 000	R=1	1.00	0.67	0.71	0.72
	R=P	1.00	0.62	0.70	0.67
V=50 000, E=250 000	R=1	1.00	0.83	1.03	1.18
	R=P	1.00	0.78	1.10	1.35

Efficiency					
V=1 000, E=5 000	R=P	100%	24%	11%	5%
V=10 000, E=50 000	R=P	100%	31%	17%	8%
V=50 000, E=250 000	R=P	100%	39%	27%	17%



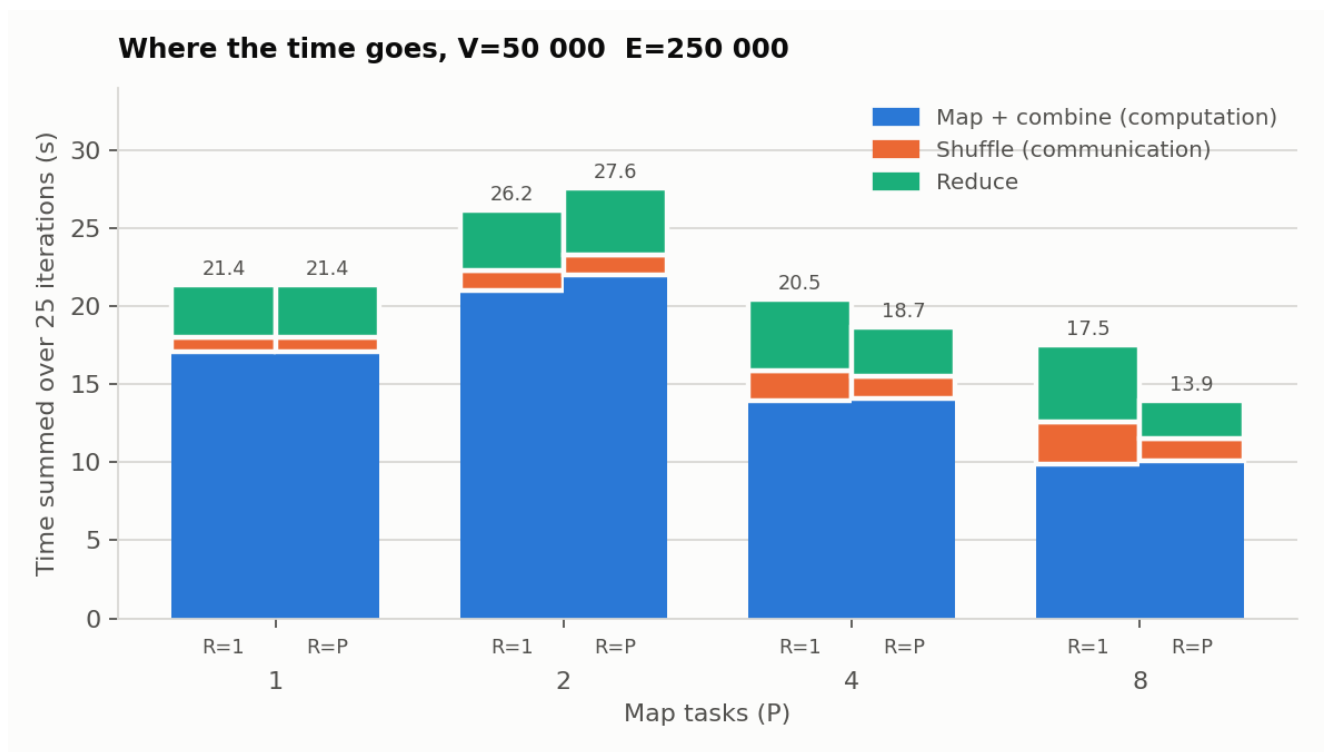


Only the largest graph exceeds its single-task time. Partitioning across R reducers raises its best speedup from 1.18× to 1.35×, and efficiency at P=8 from 15% to 17%.

Communication against computation

Per-iteration timings for the largest graph, summed over its 25 iterations:

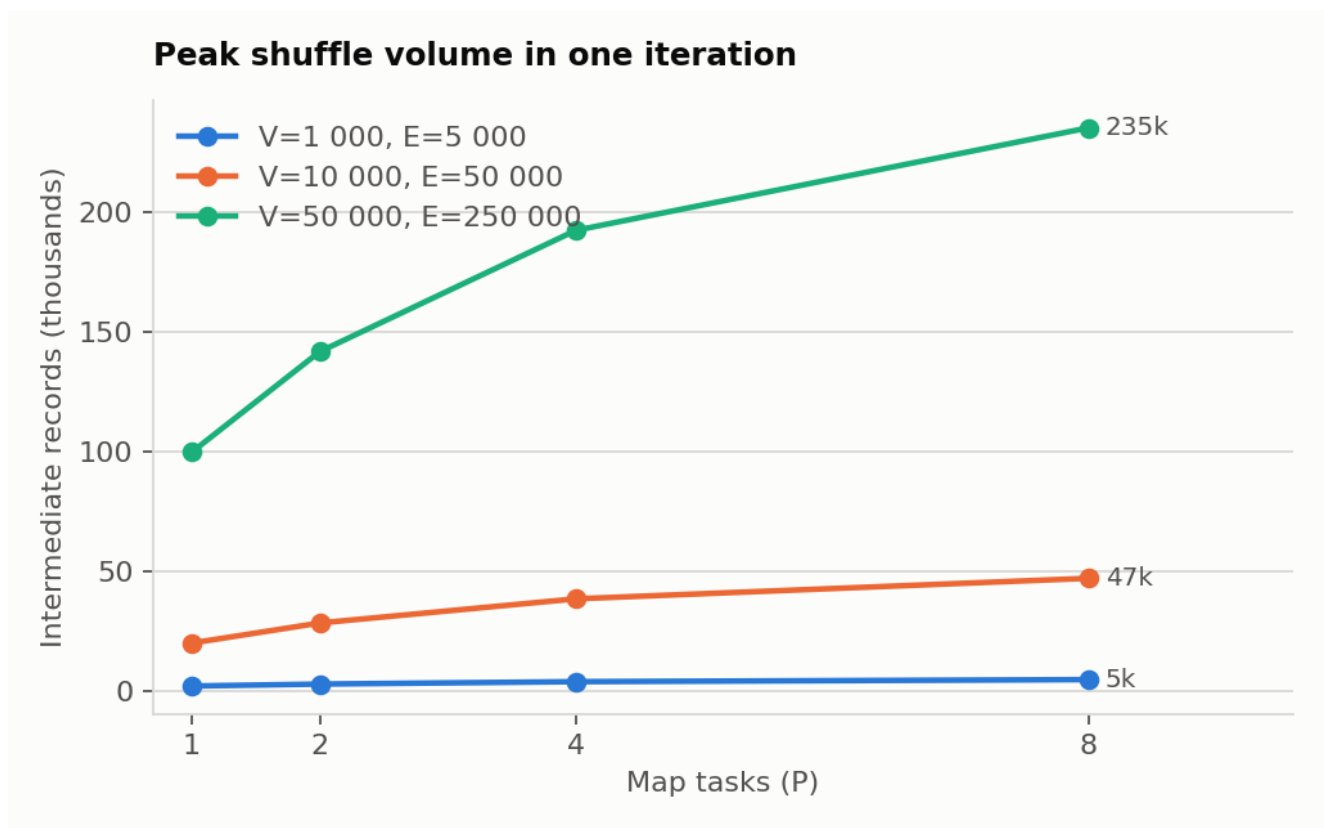
P	Reducers	Map + combine	Shuffle	Reduce
1	1	17.05 s	0.95 s	3.35 s
2	1	21.03 s	1.27 s	3.85 s
	2	21.98 s	1.31 s	4.31 s
4	1	13.96 s	1.92 s	4.57 s
	4	14.11 s	1.40 s	3.17 s
8	1	9.85 s	2.71 s	4.96 s
	8	10.09 s	1.44 s	2.39 s



With one reducer, the two stages after the map move in the wrong direction as P grows: shuffle 0.95 s to 2.71 s, reduce 3.35 s to 4.96 s. Partitioning reverses both at P=8 — shuffle falls to 1.44 s because the R merges run concurrently, and reduce to 2.39 s because the R reducers do.

Peak intermediate records crossing the shuffle in one iteration (unaffected by the reducer count, since partitioning splits the same records into more files):

Graph	P=1	P=2	P=4	P=8
V=1 000, E=5 000	1 989	2 811	3 810	4 674
V=10 000, E=50 000	19 932	28 323	38 416	46 988
V=50 000, E=250 000	99 653	141 657	192 323	235 129



5. Analysis

Communication grows with P while computation shrinks. From P=1 to P=8 the map stage falls by 7.0 s. The volume of data the shuffle must carry grows regardless of reducer count: 2.4× more records at P=8 than at P=1 for identical computation. The cause is the combiner, which runs per chunk — more chunks means each combiner sees a smaller slice, so fewer duplicate proposals meet and get merged. This is the fundamental tension in the design: dividing the map stage finer is exactly what makes the shuffle carry more.

Partitioning removes the serial tail but does not remove the data. With one reducer the shuffle and reduce stages are serial no matter how many map tasks run, and together they grow from 4.3 s to 7.7 s across the P range. Partitioning by `node_id % R` lets both run concurrently, cutting them to 3.8 s at P=8 — a 2.0× improvement on that portion. The records still have to be written, merged and read; only the serialisation goes away.

The gain is real but smaller than ideal. Eight reducers make the reduce stage 2.1× faster, not 8×. The same ceiling appears in the map stage, which reaches 2.77× at P=8 once launch overhead is subtracted. Two costs explain both: every task is a fresh Python process started once per iteration, and all tasks read and write intermediate files on `/home`, which is shared network storage rather than local disk. Writing intermediates to node-local `/scratch` is the most promising next step.

A fixed cost is paid every iteration. Each round launches the map stage with one `srun` call, timed directly by the sweep:

P	1	2	4	8
Mean empty srun	0.053 s	0.152 s	0.162 s	0.169 s

The jump is at $P=2$, then nearly flat — which is why $P=2$ is the worst configuration at every size: it pays the full overhead increase while halving only a modest amount of work. At $V=1\,000$ the actual map work is 0.03 s per iteration against 0.17 s of launch overhead, so the small graphs are measuring Slurm rather than the algorithm. This is also why $R=P$ hurts there: it adds R more process starts per iteration to a round that had almost no work to divide.

Against sequential Dijkstra the implementation is $37\times$ slower at its best (17.29 s vs 0.47 s). That is the cost of the model rather than a defect: 25 rounds, each starting fresh processes and writing the whole graph state to disk and reading it back. A graph whose state streams through 11 MB has nothing to gain from distribution.

6. Limitations

- The assignment caps V at 10 000; the sweep goes to 50 000 so the trend spans more than two points, but no further. The claim that these graphs are too small to benefit from distribution is therefore an extrapolation — showing the crossover would need graphs well outside the assignment's range.
- $P = 1$ to 8 all ran within a single node, so this measures within-node parallelism only.
- Non-negative weights only.
- Reducers run as concurrent processes inside the allocation rather than under a second **srun**. The allocation is a single node, so this is equivalent here, and a second launch per iteration would have cost more than the parallel reduce saves.
- Repeat runs of the same configuration differ by a few percent (the $P=8$, $R=1$ point was 18.96 s in an earlier sweep and 19.92 s here), so differences below roughly 5% should not be read as significant.